

Data-Migration Architecture – Cross-Industry Logic Overview

Question

How should the system's data-migration logic be structured to support banking, telecom, insurance, hospitality, and school domains, given a scraper-driven backend, a manual front-end input that can overwrite scraper data, and the need for a reliable migration path?

Approach

1. **Hypothesis** – A unified ingestion-processing-storage pipeline that treats scraper feeds and manual inputs as idempotent streams will prevent overwrites and ensure consistency across all industries.
2. **Data Model** – Define a domain-agnostic "Entity" layer (Customer, Transaction, Policy, Reservation, Student) with industry-specific extensions.
3. **Query Design** – Build a set of SQL CTEs that (a) deduplicate incoming records, (b) prioritize manual edits over scraper updates, and (c) flag conflicts for review.
4. **Validation** – Apply business-rule checks per industry (e.g., account balance ≥ 0 for banking, policy effective dates for insurance).

SQL Sketch (illustrative)

```
WITH raw_scraper AS (
    SELECT * FROM kafka_topic_scraper
),
raw_manual AS (
    SELECT * FROM api_manual_input
),
merged AS (
    SELECT
        COALESCE(m.id, s.id) AS entity_id,
        COALESCE(m.source, s.source) AS source,
        COALESCE(m.updated_at, s.updated_at) AS ts,
        CASE WHEN m.id IS NOT NULL THEN m.* ELSE s.* END AS payload
    FROM raw_scraper s
    FULL OUTER JOIN raw_manual m
        ON s.id = m.id
),
validated AS (
    SELECT *
    FROM merged
    WHERE
        -- Banking rule
        (entity_type = 'banking' AND payload.balance >= 0) AND
        -- Telecom rule
        (entity_type = 'telecom' AND payload.msisdn IS NOT NULL) AND
        -- Insurance rule
        (entity_type = 'insurance' AND payload.policy_end > CURRENT_DATE) AND
        -- Hospitality rule
```

```

        (entity_type = 'hospitality' AND payload.check_in <= payload.check_out) AND
        -- Schools rule
        (entity_type = 'school'      AND payload.enrollment_status IN ('active','inactive'
    )
    INSERT INTO unified_entity (entity_id, source, ts, payload)
    SELECT entity_id, source, ts, payload
    FROM validated
    ON CONFLICT (entity_id) DO UPDATE
        SET source = EXCLUDED.source,
            ts      = EXCLUDED.ts,
            payload= EXCLUDED.payload;

```

Key Findings

- **Idempotent upserts** (ON CONFLICT) guarantee that manual edits overwrite scraper data without duplication.
- **Unified domain model** reduces schema drift across industries and simplifies downstream analytics.
- **Real-time streaming (Kafka) + API layer** provides low-latency ingestion while preserving auditability.
- **Validation layer** catches industry-specific anomalies early, lowering downstream error rates.

Caveats

- The “fintech logic” folder was not found on the local machine (fs.find_in returned 0 matches) → any proprietary transformation rules stored there could not be incorporated.
- Assumes source data is normalized to the unified entity schema; additional ETL mapping may be required for legacy systems.

Recommended Action

Implement the outlined ingestion pipeline, starting with a pilot for the banking and telecom domains to validate the upsert logic and validation rules. Extend the same pattern to insurance, hospitality, and schools once the core flow is stable.

Assumed: the Architecture Summary Report contains the high-level system components referenced in the email sent (see evidence [5]); no additional domain-specific documents were available.